



Memoria Tecnica - Proyecto Final Programacion

Liga de E-Sports

Autor: Pablo Nunez Fernandez

Modulo: Programacion - 1 CFGS DAW

Fecha: Mayo 2026

1. Descripcion general del proyecto

La aplicacion gestiona una liga de e-sports de 6 equipos al estilo MOBA (League of Legends). Permite registrar jugadores y entrenadores, formar plantillas, programar y disputar partidos, registrar incidencias y sanciones, y

consultar la clasificacion final con estadisticas.

Todo funciona en memoria (sin ficheros ni base de datos). Al arrancar se cargan datos de ejemplo con 6 equipos completos para poder probar todas las funcionalidades desde el primer momento.

2. Estructura de paquetes

```
src/
├── Main.java           -> punto de entrada, menus e inicializacion de datos
├── model/              -> clases del dominio (entidades de la liga)
│   ├── PersonaLiga.java -> clase abstracta base
│   ├── Jugador.java    -> extiende PersonaLiga, implementa Entrenable
│   ├── Entrenador.java -> extiende PersonaLiga
│   ├── Entrenable.java -> interfaz
│   ├── Equipo.java     -> gestion de plantilla (array + ArrayList)
│   ├── Partido.java    -> resultado y estadisticas
│   ├── Incidencia.java -> sanciones y otros sucesos
│   ├── Rol.java        -> enum con los 5 roles
│   └── TipoIncidencia.java -> enum con los tipos de incidencia
├── exceptions/         -> excepciones personalizadas
│   ├── RolNoDisponibleException.java
│   └── JugadorSancionadoException.java
└── structures/
    └── Liga.java        -> clase central con todas las estructuras de datos
```

3. Diseno de clases

3.1 Jerarquia de herencia

```
PersonaLiga (abstracta)
├── Jugador (implementa Entrenable)
└── Entrenador
```

PersonaLiga es abstracta porque define los atributos y comportamientos comunes a todos los miembros de la liga (ID, nombre, nickname, edad, salario base), pero el metodo `calcularCosteMensual()` lo declara abstracto porque cada tipo de persona lo calcula de forma diferente.

Jugador ania: rol, niveles mecanico y estrategico, partidas jugadas, MVPs y estado de sancion. Implementa la interfaz Entrenable.

Entrenador ania: experiencia, especialidad tactica y victorias totales.

3.2 Interfaz Entrenable

```
public interface Entrenable {  
    void entrenar();  
    double calcularRendimiento();  
}
```

La implementa Jugador. El metodo `calcularRendimiento()` usa una formula ponderada:

Componente	Peso	Calculo
Nivel mecanico	40%	nivelMecanico x 0.4
Nivel estrategico	40%	nivelEstrategico x 0.4
Ratio MVP	20%	(mvpTotales / partidasJugadas) x 10 x 0.2

Si el jugador no ha disputado ninguna partida, el componente MVP es 0 para evitar una division por cero.

3.3 Clase Equipo - doble estructura de jugadores

Cada equipo tiene dos niveles de jugadores:

- **Jugador[] titulares** - array fijo de 5 posiciones. Cada indice corresponde al ordinal del enum Rol (0=TOP, 1=JUNGLE, 2=MID, 3=ADC, 4=SUPPORT). Si una posicion es null, ese rol esta vacante.
- **ArrayList suplentes** - lista dinamica sin restriccion de rol.

Esta doble estructura es obligatoria segun el enunciado del proyecto. El array fijo garantiza eficiencia y unicidad de rol en los titulares. La lista dinamica permite gestionar suplentes sin saber cuantos habra.

Regla de unicidad de rol: al fichar un titular se comprueba `titulares[jugador.getRol().ordinal()]`. Si ya hay un jugador en esa posicion, se lanza `RolNoDisponibleException`.

3.4 Calculos de coste mensual

Clase	Formula
Jugador	salarioBase + (nivelMecanico + nivelEstrategico) x 100
Entrenador	salarioBase + (experiencia x 200)
Equipo.calcularCosteEquipo()	Suma de todos los jugadores + entrenador

3.5 Clase Partido - registro de resultados

Un partido se crea en estado pendiente (jugado = false). Al llamar a `registrarResultado()` se:

1. Guarda el marcador y el MVP.
2. Llama a `registrarVictoria()` / `registrarDerrota()` / `registrarEmpate()` en ambos equipos.
3. Incrementa el contador de partidas de todos los titulares.
4. Incrementa el contador de MVPs del jugador designado.

5. Marca el partido como jugado.

Si se intenta registrar un partido ya jugado, se lanza `IllegalStateException`.

3.6 Enumeraciones

Rol - 5 valores: TOP, JUNGLE, MID, ADC, SUPPORT. Su ordinal (0-4) se usa directamente como indice en el array `titulares[]` de Equipo.

TipoIncidencia - 5 valores: SANCION, EXPULSION, ERROR_TECNICO, PARTIDO_APLAZADO, OTRO.

3.7 Clase Liga - clase central

Contiene y coordina todos los elementos de la liga. Es el unico punto donde se crean y registran equipos, personas y partidos.

4. Estructuras de datos dinamicas - justificacion

4.1 ArrayList

Se usan cuatro listas dinamicas en Liga:

Lista	Contenido	Motivo
ArrayList	Equipos de la liga	Necesitamos su indice para la matriz de puntos
ArrayList	Jugadores y entrenadores	Permite tratar ambos tipos de forma polimorfica
ArrayList	Historial completo de partidos	Acceso aleatorio para buscar por jornada
ArrayList	Incidencias registradas	Numero desconocido a priori

Tambien en Equipo.suplentes: el numero de suplentes es variable, por lo que no se puede usar un array fijo.

4.2 HashSet - control de duplicados

```
HashSet<String> idsPersonas;  
HashSet<String> nombresEquipos;  
HashSet<String> idsPartidos;
```

Se usan tres HashSet para detectar duplicados en $O(1)$ antes de insertar. Un ArrayList requeriria recorrer toda la lista ($O(n)$) para comprobar si ya existe un elemento. Con HashSet, la comprobacion `contains()` es constante independientemente del numero de elementos.

4.3 Queue (Cola FIFO) - partidos pendientes

```
Queue<Partido> colaPartidos; // implementada con LinkedList
```

La cola garantiza que los partidos se disputan en el orden en que fueron registrados (FIFO: First In, First Out). El primer partido anadido es el primero en jugarse. Se usa `peek()` para ver el proximo sin sacarlo, y `poll()` para sacarlo al jugarlo.

4.4 ArrayDeque como Pila (LIFO) - historial de acciones

```
ArrayDeque<String> pilaAcciones;
```

Cada vez que el usuario realiza una operacion (registrar persona, crear equipo, aplicar sancion...) se anade una descripcion a la cima de la pila con `push()`. Al mostrar el historial, las acciones mas recientes aparecen primero (LIFO: Last In, First Out). Se usa `peek()` para ver la ultima sin eliminarla.

4.5 ArrayDeque como Pila - deshacer sanciones

```
ArrayDeque<String> pilaSanciones;
```

Pila auxiliar que guarda los IDs de los jugadores sancionados. Al deshacer, se extrae el ID de la cima con `pop()` y se revierte la sancion del jugador correspondiente (comportamiento LIFO: se deshace la ultima sancion aplicada).

4.6 Matriz 2D - puntos por jornada

```
int[][] matrizPuntos = new int[NUM_JORNADAS][MAX_EQUIPOS]; // [5][6]
```

Almacena los puntos acumulados por cada equipo en cada jornada:

- **Filas** (indice 0-4): jornadas de la liga
- **Columnas** (indice 0-5): equipos, por orden de insercion

Puntuacion: victoria = 3 puntos, empate = 1 punto, derrota = 0 puntos.

Permite consultar la evolucion de puntos jornada a jornada, y se actualiza automaticamente cada vez que se registra el resultado de un partido.

5. Excepciones personalizadas

5.1 RolNoDisponibleException

```
public class RolNoDisponibleException extends Exception
```

Cuando se lanza: al intentar fichar un jugador como titular con un rol que ya esta ocupado en el equipo, o al intercambiar titular y suplente con roles incompatibles.

Donde se lanza: Equipo.ficharTitular() y Equipo.intercambiarTitularSuplente()

Donde se captura: Main.ficharTitular() y Main.intercambiarJugadores()

5.2 JugadorSancionadoException

```
public class JugadorSancionadoException extends Exception
```

Cuando se lanza: al intentar fichar como titular o suplente a un jugador con sancionado = true.

Donde se lanza: Jugador.verificarDisponibilidad(), llamado desde Equipo.ficharTitular(), Equipo.ficharSuplente() e Equipo.intercambiarTitularSuplente()

Donde se captura: Main.ficharTitular(), Main.ficharSuplente() y Main.intercambiarJugadores()

Como se revierte: mediante la operacion undo (opcion 11 del menu), que extrae el ID de la pila pilaSanciones y llama a jugador.setSancionado(false).

6. Menu de la aplicacion

Opcion	Funcionalidad
1	Gestion de personas (aniadir jugador/entrenador, buscar, modificar, eliminar)
2	Gestion de equipos (crear, listar, ver plantilla, asignar entrenador)
3	Fichajes y convocatorias (fichar titular/suplente, intercambiar, eliminar)
4	Calendario (registrar partido, ver partidos, ver matriz de puntos)
5	Cola de partidos pendientes (ver proximo, ver cola, limpiar)
6	Registrar resultado del proximo partido de la cola
7	Incidencias y sanciones (registrar, aplicar sancion, buscar)
8	Mostrar clasificacion final
9	Estadisticas de jugadores y costes de equipos
10	Historial de acciones (pila LIFO)
11	Deshacer ultima sancion (undo)
12	Salir

7. Flujo principal de uso

- 1. Al arrancar se cargan 6 equipos con plantillas completas (inicializarDatos).
- 2. El usuario puede registrar mas personas y equipos desde el menu.
- 3. Los partidos se registran en el calendario y pasan automaticamente a la cola.
- 4. Desde la opcion 6 se juegan los partidos en orden FIFO: se introduce el resultado y el MVP, y se actualiza la matriz de puntos.
- 5. Las sanciones se aplican desde incidencias (opcion 7) y se deshacen con la opcion 11.
- 6. La clasificacion (opcion 8) se calcula en tiempo real desde los datos almacenados.

8. Datos de ejemplo cargados al inicio

Equipo	Ciudad	Entrenador
Storm Dragons	Madrid	CarlosCoach (10 anios, Macro)
Night Wolves	Barcelona	LauraG (7 anios, Teamfight)
Iron Phoenix	Valencia	MarcosCoach (15 anios, Draft)
Azure Hawks	Sevilla	SofiaV (5 anios, Early Game)
Silver Ravens	Bilbao	DiegoCoach (8 anios, Jungla)
Gold Falcons	Zaragoza	AnaCoach (6 anios, Support)

Cada equipo tiene 5 titulares (uno por rol) + 1 suplente. En total: 36 jugadores, 6 entrenadores, 6 partidos de jornadas 1 y 2 en la cola.

9. IDs de referencia para pruebas

Rango	Tipo
E001 - E006	Entrenadores
J101 - J106	Storm Dragons
J201 - J206	Night Wolves
J301 - J306	Iron Phoenix
J401 - J406	Azure Hawks
J501 - J506	Silver Ravens
J601 - J606	Gold Falcons
P001 - P006	Partidos de jornadas 1 y 2